SafeRide Auth Service — Interview Questions & Answers

> A comprehensive Q&A covering architecture decisions, OAuth 2.0, OTP, SSO, security hardening, and real implementation details from the SafeRide auth codebase.

Table of Contents

1. Architecture & High-Level Design
2. OAuth 2.0 & OIDC Core Concepts
3. PKCE (Proof Key for Code Exchange)
4. OTP (One-Time Password) Security
5. Cookie Security & Session Management
6. Registration Flow Security
7. Login Flow Security
8. SSO (Single Sign-On)
9. Token Management & Lifecycle
10. Consent Flow & Hijack Prevention
11. CSRF, State & Nonce Protection
12. Session Fixation & Replay Attacks
13. Open Redirect Prevention
14. Distributed System Compensation (Saga)
15. Rate Limiting & Brute Force Protection
16. Security Headers
17. Google OAuth / Social Login
18. Refresh Token Rotation & Grace Period
19. Ory Stack (Kratos, Hydra, Keto)
20. Outbox Pattern & Event-Driven Design
21. Database & Migration Design
22. Logout & Token Revocation
23. Attack Scenarios & Mitigations
24. Production Readiness

1. Architecture & High-Level Design

Q1.1: What is the overall architecture of the SafeRide Auth Service?

Answer: The SafeRide Auth Service follows a **Clean/Hexagonal Architecture** with distinct layers:

Layer	Directory	Responsibility
API	`internal/api/`	HTTP handlers, DTOs, middleware, routes, validators
Application	`internal/application/`	Business logic orchestration (login, register, refresh, logout, cleanup)
Domain	`internal/domain/`	Core entities & repository interfaces (user, pendingauth, oauthsession, events)
Infrastructure	`internal/infrastructure/`	Concrete implementations (Postgres, Redis, Kafka, Hydra, Kratos, Keto clients)
Platform	`internal/platform/`	Cross-cutting abstractions (cache, eventbus, identity orchestrator)
Shared	`internal/shared/`	Utilities (OTP, crypto, constants, error types, response helpers)

The service integrates with **Ory Hydra** (OAuth2/OIDC provider), **Ory Kratos** (identity management), **Ory Keto** (authorization/permissions), **Redis** (session cache, rate limiting), **PostgreSQL** (persistent storage), and **Kafka** (event bus via outbox pattern).

Q1.2: Why did you choose this layered architecture instead of a simpler MVC pattern?

Answer: The layered architecture provides:

- **Testability** — Domain and application layers are framework-agnostic, making unit testing straightforward without HTTP dependencies.
- **Dependency Inversion** — The domain defines repository interfaces; infrastructure implements them. Swapping PostgreSQL for another DB requires zero changes in business logic.
- **Separation of Concerns** — The handler layer only handles HTTP request/response parsing and delegates to application services, which orchestrate domain logic and external calls.

- **Maintainability** — Each layer has a single responsibility. Adding a new auth flow (e.g., WebAuthn/FIDO2) only requires new application + handler code without touching existing domain or infrastructure.

Q1.3: Why use a Backend-for-Frontend (BFF) pattern where the backend manages all OAuth redirects?

Answer: The frontend **never** touches tokens directly. The backend acts as a BFF (Backend-for-Frontend):

- **Tokens are stored in HttpOnly cookies**, making them inaccessible to JavaScript (prevents XSS-based token theft).
- **PKCE code_verifier stays server-side** — it never leaves the backend, so even if a browser extension sniffs the authorization code, it cannot exchange it.
- **All redirects pass through backend validation** — the backend validates redirect URLs against a whitelist before issuing 302 responses, preventing open redirect attacks.

If we exposed tokens to the frontend (e.g., `localStorage`), any XSS vulnerability would give an attacker full access to the user's tokens.

---

2. OAuth 2.0 & OIDC Core Concepts

Q2.1: Why did you choose OAuth 2.0 Authorization Code Flow instead of Implicit Flow or Client Credentials?

Answer:

- **Implicit Flow** is deprecated by OAuth 2.1 because it exposes tokens in URL fragments (browser history, server logs, referrer headers). It offers no PKCE or refresh token support.
- **Client Credentials** is for machine-to-machine communication, not for user-facing authentication.
- **Authorization Code Flow** keeps access tokens out of the browser URL. Combined with PKCE, it's the most secure flow for web applications. The authorization code is short-lived (typically 10 minutes) and single-use — Hydra invalidates it after exchange.

Q2.2: What is OIDC and how does it differ from plain OAuth 2.0?

Answer:

- **OAuth 2.0** is an **authorization** framework — it grants access tokens to resources but doesn't tell you *who* the user is.
- **OIDC (OpenID Connect)** is an **identity layer on top of OAuth 2.0** — it adds an `id_token` (a signed JWT) that contains user identity claims (subject, email, etc.).

In our implementation, we request the `openid` scope so Hydra returns an `id_token` alongside the `access_token`. We then:

1. Fetch Hydra's JWKS (JSON Web Key Set) public keys.
2. Verify the `id_token` 's **RS256 signature** against JWKS.
3. Verify the **nonce** inside the `id_token` matches the one stored in our session (replay protection).

This guarantees the `id_token` wasn't forged or replayed.

Q2.3: What scopes do you request and why?

Answer:

```
openid + offline_access
```

- **`openid`** — Required for OIDC; triggers Hydra to issue an `id_token` .
- **`offline_access`** — Requests a `refresh_token` , allowing silent re-authentication without user interaction. Without this scope, after the access token expires, the user would have to re-login.

Q2.4: Explain the full token exchange flow step by step.

Answer:

1. **Backend generates** `state` , `nonce` , `PKCE code_verifier` + `code_challenge` , and stores them in a DB-backed `oauth_sessions` table.
2. **Backend redirects** browser to Hydra's `/oauth2/auth` with `client_id` , `redirect_uri` , `state` , `nonce` , `code_challenge` , and `code_challenge_method=S256` .
3. **Hydra creates a login_challenge** → redirects to our backend's login-challenge endpoint.
4. **Backend binds login_challenge** to the session and either serves the login UI or auto-accepts (SSO/registration).
5. **User authenticates** (via OTP or auto-accept) → backend calls Hydra's `PUT /admin/oauth2/auth/requests/login/accept` .
6. **Hydra issues consent_challenge** → backend auto-accepts consent → Hydra issues the authorization `code` .
7. **Hydra redirects to callback** → backend validates `state` matches session, then calls `POST /oauth2/token` with `{code, code_verifier}` .
8. **Hydra validates PKCE** (`SHA256(code_verifier) == code_challenge`), and returns `access_token` , `refresh_token` , `id_token` .
9. **Backend validates** `id_token` signature (RS256 via JWKS) and nonce.
10. **Backend sets tokens** as HttpOnly cookies and redirects to `/home` .

3. PKCE (Proof Key for Code Exchange)

Q3.1: What is PKCE and why is it necessary?

Answer: PKCE (RFC 7636) protects against **authorization code interception attacks**. Without PKCE, if an attacker intercepts the authorization code (via a malicious browser extension, open redirect, or network MITM), they can exchange it for tokens.

With PKCE:

- Backend generates a cryptographic `code_verifier` (random 32 bytes, base64url-encoded).
- Backend computes `code_challenge = BASE64URL(SHA256(code_verifier))`.
- `code_challenge` is sent to Hydra; `code_verifier` **stays server-side** (never sent to the browser).
- At token exchange, backend sends `code_verifier`. Hydra recomputes `SHA256(code_verifier)` and checks it matches the originally stored `code_challenge`.

An attacker who intercepts the code **cannot produce the code_verifier**, so the token exchange fails.

Q3.2: Show me exactly how PKCE is implemented in the code.

Answer:

```
// Generate a 32-byte random verifier (base64url-encoded, ~43 chars)
func generateVerifier() (string, error) {
    b := make([]byte, 32)
    if _, err := rand.Read(b); err != nil {
        return "", err
    }
    return base64.RawURLEncoding.EncodeToString(b), nil
}

// Compute S256 challenge: SHA-256 hash of verifier, base64url-encoded
func generateChallenge(verifier string) string {
    h := sha256.New()
    h.Write([]byte(verifier))
    return base64.RawURLEncoding.EncodeToString(h.Sum(nil))
}
```

The verifier is stored in `oauth_sessions.code_verifier` (database, never sent to the browser). During callback, it's sent to Hydra's token endpoint:

```
tokens, err := h.orchestrator.ExchangeCodeForTokens(ctx, code, session.CodeVerifier)
```

Q3.3: Why S256 instead of plain PKCE?

Answer: With **plain** PKCE, the `code_challenge` IS the `code_verifier` — if an attacker sees the challenge in the authorization URL, they know the verifier. With **S256**, the challenge is a one-way hash of the verifier. Even if the attacker intercepts the challenge, they cannot reverse-engineer the verifier.

---

4. OTP (One-Time Password) Security

Q4.1: How is the OTP generated and how do you ensure it's cryptographically secure?

Answer:

```
func GenerateOTP() (string, error) {
    max := big.NewInt(900000)
    n, err := rand.Int(rand.Reader, max)
    // ...
    otpVal := n.Add(n, big.NewInt(100000)).String()  // 100000–999999
    return otpVal, nil
}
```

- Uses `crypto/rand.Reader` (CSPRNG — Cryptographically Secure Pseudo-Random Number Generator), NOT `math/rand`.
- `math/rand` is seeded with a predictable value (often timestamp), so an attacker could predict OTPs. `crypto/rand` sources entropy from the OS kernel (`/dev/urandom` on Linux, `CryptGenRandom` on Windows).
- Generates a 6-digit number in range [100000, 999999] — never starts with 0, ensuring consistent length.

Q4.2: Why do you hash the OTP before storing it? Why not store it plaintext?

Answer:

```
func HashOTP(otp string) string {
    hash := sha256.Sum256([]byte(otp))
    return hex.EncodeToString(hash[:])
}
```

If the database is compromised (SQL injection, backup leak, insider threat), an attacker would see **OTP hashes**, not cleartext OTPs. Since OTPs are 6-digit numbers (only 900,000 possibilities), SHA-256 alone isn't ideal for offline brute-

force — but OTPs are **time-limited (5-minute TTL)** and **attempt-limited (max attempts)**, making offline brute-force impractical. By the time an attacker gets the hash, the OTP has expired.

The cleartext OTP is only used once — to construct the email payload — and is never persisted in the database.

Q4.3: What is a timing attack and how does constant-time comparison prevent it?

Answer: A **timing attack** exploits the fact that standard string comparison (`==`) short-circuits — it returns `false` immediately when the first mismatched character is found. An attacker can submit OTPs and measure response times:

- `123456` vs `654321` → mismatch on 1st char → fast response (e.g., 1ms)
- `623456` vs `654321` → mismatch on 2nd char → slightly slower (e.g., 1.1ms)
- `653456` vs `654321` → mismatch on 3rd char → even slower (e.g., 1.2ms)

By correlating response times, the attacker can brute-force one digit at a time (6 × 10 = 60 attempts instead of 10^6).

Our implementation uses `crypto/subtle.ConstantTimeCompare`:

```
func VerifyOTP(otp, hashedOTP string) bool {
    return subtle.ConstantTimeCompare([]byte(HashOTP(otp)), []byte(hashedOTP)) == 1
}
```

This compares **every byte** regardless of where a mismatch occurs, taking the same amount of time for any input — eliminating the timing side channel.

Q4.4: What other brute-force protections exist for OTP?

Answer: We employ **defense in depth** with three layers:

Protection	Implementation
Attempt Limiting	`pending_auth.attempt_count` tracked per session. After `max_attempts` (configurable, default 5), the session is deleted and returns `403 Forbidden`.
Time Expiry	`pending_auth.expires_at` set to `now + 5 minutes`. Expired OTPs are rejected and cleaned up.
IP-based Rate Limiting	Redis-backed rate limiter on login/register endpoints. Exceeding the limit returns `429 Too Many Requests` with `Retry-After` header.

```
// Attempt check
if pa.HasExceededAttempts() {
    _ = s.pendingAuthRepo.Delete(ctx, pa.ID)
    return nil, apperrors.Forbidden("Too many failed attempts", ...)
}

// On wrong OTP
if !otp.VerifyOTP(req.OTP, pa.OTPHash) {
    pa.IncrementAttempts()
    _ = s.pendingAuthRepo.Update(ctx, pa)
    // ...
}
```

---

5. Cookie Security & Session Management

Q5.1: What cookie attributes do you use and why each one?

Answer:

Attribute	Value	Why
HttpOnly	`true`	Prevents JavaScript access (`document.cookie`). Mitigates XSS-based token theft.
Secure	`true` in production	Cookies only sent over HTTPS. Prevents interception over plain HTTP.
SameSite	`Lax`	Prevents CSRF — cookies are NOT sent on cross-site POST requests. `Lax` allows top-level navigations (redirects) which is required for OAuth flows.
Path	Scoped per cookie	`access_token` → `/` (needs wide access), `refresh_token` → `/api/v1/auth` (only auth endpoints), `session_id` → `/api/v1/auth` (only auth endpoints). Minimizes cookie exposure.

```
func (h *OAuthHandler) setCookie(c *gin.Context, name, value string, maxAge int, path st
    c.SetSameSite(http.SameSiteLaxMode)
    c.SetCookie(name, value, maxAge, path, "", h.isProduction, true)
}
```

Q5.2: Why is `SameSite=Lax` used instead of `Strict`?

Answer: OAuth 2.0 Authorization Code Flow involves **cross-origin redirects** (Hydra → Backend → Hydra → Backend). With `SameSite=Strict`, cookies wouldn't be sent during these top-level navigations because the request originates from a different domain (Hydra). `Lax` allows cookies on top-level GET navigations (redirects) but blocks them on cross-site POST/PUT/DELETE requests — providing CSRF protection while still supporting the OAuth redirect chain.

Q5.3: Why is the `login_challenge` cookie set as non-HttpOnly?

Answer: The `login_challenge` cookie is intentionally `HttpOnly=false` because the React frontend needs to read it:

```
c.SetCookie("login_challenge", challenge, 3600, "/", "", h.isProduction, false)
```

The frontend uses the **Flash Cookie Pattern**:

1. React mounts on `/login`.
2. Reads `login_challenge` from `document.cookie` into memory.
3. **Immediately deletes the cookie** (one-time read).
4. Sends `login_challenge` with the API request.

This minimizes the window of exposure. The `login_challenge` is not a secret — it's a reference ID that can only be used by the session-bound backend to call Hydra Admin APIs.

Q5.4: What cookies does the system manage and what is each one's purpose?

Answer:

Cookie	Purpose	HttpOnly	Path	TTL
`session_id`	Binds browser to the OIDC OAuth session (state, nonce, PKCE stored in DB)	✅	`/api/v1/auth`	1 hour
`registration_session_id`	Binds browser to the registration flow (Redis-backed)	✅	`/api/v1/auth`	5 min
`login_session_id`	Binds browser to the direct login flow (Redis-backed)	✅	`/api/v1/auth`	5 min
`login_challenge`	Passes Hydra's challenge to React frontend	❌	`/`	1 hour
`access_token`	Bearer token for API authentication	✅	`/`	Token expiry
`refresh_token`	Used to obtain new access tokens	✅	`/api/v1/auth`	24 hours
`oauthstate`	Google OAuth CSRF protection	✅	`/api/v1/auth/google`	10 min
`oauth_intent`	Signals registration intent during OIDC flow	✅	`/`	5 min

6. Registration Flow Security

Q6.1: Walk through the registration flow and explain each security check.

Answer: The registration flow has **4 phases**:

Phase 1 — Initiation:

- Backend generates a `registration_session_id` (UUID), stores `"active"` in Redis with 5-min TTL.
- Sets it as an HttpOnly cookie.
- This creates a **browser-binding** — only the browser that started registration can complete it.

Phase 2 — Form Submission:

- Backend reads `registration_session_id` cookie and verifies it exists in Redis.
- Generates OTP, hashes it, stores in `pending_auth` table **along with the registration_session_id in metadata**.
- Returns `requestId` to the frontend.

Phase 3 — OTP Verification:

- Backend reads the `registration_session_id` cookie again and verifies Redis.
- Performs **Double-Binding Check**: asserts `pending_auth.metadata.registration_session_id == cookie value`. This ensures the browser completing the OTP is the same one that initiated registration — preventing an attacker from stealing the `requestId` and submitting OTP from a different browser.
- Verifies OTP with constant-time comparison.
- Creates identity in Kratos + user in Postgres + relations in Keto — all within a DB transaction with compensating actions on failure.

Phase 4 — Auto-Login Transition:

- Stores `{kratos_id, verified: true, authenticated: true}` in the Redis registration session.
- Returns `{redirectTo: "/api/v1/auth/oauth/login"}`.
- When `/oauth/login` is called with the `registration_session_id` cookie, the backend sees the verified session and auto-accepts the Hydra login challenge without showing the login form.

Q6.2: What is the "Double-Binding Check" and what attack does it prevent?

Answer: The Double-Binding Check ensures:

```
if meta.RegistrationSessionID ≠ req.RegistrationSessionID {
    return nil, apperrors.Forbidden("Session hijacking attempt detected", nil)
}
```

Attack scenario without it: An attacker obtains a valid `requestId` (e.g., via shoulder surfing or phishing). They attempt to submit the OTP from their own browser. Without the double-binding check, the server would accept the OTP and create the account, giving the attacker control.

With the check: The `requestId` is bound to a specific `registration_session_id`, which is stored in an HttpOnly cookie. The attacker's browser doesn't have this cookie, so the server-side metadata check fails with `403 Forbidden`.

Q6.3: What happens if the Kratos identity is created but the database transaction fails?

Answer: We implement a **Saga-style compensation pattern**:

```
compensate := func() {
    if isNewIdentity {
        _ = s.orchestrator.DeleteIdentity(ctx, kratosID)
    }
}

// Begin DB transaction...
if err := s.userRepo.Create(ctxTx, u); err ≠ nil {
    tx.Rollback()
    compensate()  // ← Delete the Kratos identity to maintain consistency
    return nil, err
}
```

This ensures **no orphaned Kratos identities** exist without corresponding database records. The compensation only runs for **newly created** identities — if we reused an existing Kratos identity (Case A: email already exists in Kratos), we don't delete it because other services may depend on it.

---

7. Login Flow Security

Q7.1: Explain the standard OIDC login flow with all security checks.

Answer:

1. **SSO Check (Fast-Path):** If the browser has an `access_token` cookie, introspect it via Hydra. If active, redirect directly to `/home` — no login needed.
2. **Session Creation:** Generate `session_id` (UUID), `state` (CSRF), `nonce` (replay), PKCE pair. Store in `oauth_sessions` table. Set `session_id` as HttpOnly cookie.
3. **Hydra Redirect:** Browser is redirected to Hydra's `/oauth2/auth` endpoint with all parameters.
4. **Login Challenge:** Hydra creates `login_challenge` → redirects to our `/login-challenge` endpoint. We:
    - Validate `session_id` cookie (exists, not expired, not used).
    - Verify challenge is real via Hydra Admin API (`GET /admin/oauth2/auth/requests/login`).
    - Bind `login_challenge` to the `session_id` in the DB.
    - Check for auto-login (registration session, login session, or SSO fallback).
    - Set `login_challenge` cookie for the React frontend.

5. **User Authentication:** User enters email → backend validates session → generates OTP → sends email. User enters OTP → constant-time verification → session regeneration (new `session_id`).
6. **Challenge Acceptance:** Backend calls `PUT /admin/oauth2/auth/requests/login/accept` with `{subject: KratosID}`.
7. **Consent:** Auto-accepted with hijack prevention check (see Section 10).
8. **Callback (Double-Lock):**
   - Read `session_id` cookie → find session in DB.
   - Compare `state` from URL query with `state` from session.
   - Check `used == false`.
   - Exchange code with PKCE `code_verifier`.
   - Validate `id_token` signature (RS256 via JWKS) and nonce.
   - Mark session as used, delete it, clear cookies.
   - Set `access_token` and `refresh_token` cookies.

Q7.2: What is the "Double-Lock Check" in the callback and why is it important?

Answer: The Double-Lock Check combines **two independent verification mechanisms**:

```
// Lock 1: Cookie binding — proves it's the same browser
session, err := h.readAndValidateSession(c)

// Lock 2: State validation — proves it's the same OAuth flow
if session.State ≠ state {
    return error("State parameter mismatch")
}
```

Why two locks?

- **Cookie alone** is insufficient — if an attacker could set a cookie in the victim's browser (cookie tossing attack), they could hijack the session.
- **State alone** is insufficient — the state parameter travels through the browser URL, which could be intercepted (browser history, referrer leaks, server logs).
- **Combined**, an attacker would need to both intercept the state AND have the session cookie, which is protected by HttpOnly + SameSite + Secure.

Additionally, the `used` flag prevents **replay attacks** — even if all parameters are intercepted, they can only be used once.

Q7.3: Why do you regenerate the session_id after successful OTP verification?

Answer: This prevents **session fixation attacks**.

Attack scenario: An attacker generates a valid `session_id` (perhaps by initiating the OAuth flow themselves) and tricks the victim into using it (e.g., by injecting the cookie). After the victim authenticates, the attacker knows the `session_id` and can use it to complete the OAuth flow in their own browser.

Mitigation:

```
// After successful OTP verification:
newSessionID := uuid.New()
newSession := &oauthsession.OAuthSession{
    ID:             newSessionID,
    State:          session.State,     // preserve cryptographic material
    Nonce:          session.Nonce,
    CodeVerifier:   session.CodeVerifier,
    LoginChallenge: session.LoginChallenge,
    Used:           false,
    // ...
}
h.oauthSessionRepo.Create(ctx, newSession)   // create new
h.oauthSessionRepo.Delete(ctx, session.ID)   // delete old
h.setCookie(c, "session_id", newSessionID.String(), ...)  // update cookie
```

The attacker's old `session_id` is invalidated. The victim's browser gets the new one. The attacker can no longer use their known `session_id`.

---

8. SSO (Single Sign-On)

Q8.1: How does SSO work in this system?

Answer: SSO provides seamless authentication across multiple applications. It works at **three levels**:

Level 1 — Fast-Path (at `/oauth/login`):

```
token, err := c.Cookie("access_token")
if err == nil && token != "" {
    active, _, _, introErr := h.orchestrator.IntrospectToken(ctx, token)
    if introErr == nil && active {
        c.Redirect(http.StatusFound, h.frontendURL+"/home")
        return
    }
}
```

If the user already has a valid `access_token` cookie, skip the entire OAuth flow and redirect directly to `/home`.

Level 2 — Fallback (at `/login-challenge`): Even if the Fast-Path was missed (e.g., different subdomain), when Hydra redirects to our login challenge endpoint, we check again:

```
token, tokenErr := c.Cookie("access_token")
if tokenErr == nil && token != "" {
    active, subject, _, introErr := h.orchestrator.IntrospectToken(ctx, token)
    if introErr == nil && active && subject != "" {
        redirectTo, acceptErr := h.orchestrator.AcceptLoginChallenge(ctx, challenge, sub
        c.Redirect(http.StatusFound, redirectTo)
        return
    }
}
```

This auto-accepts the login challenge using the existing token's subject, completely bypassing the login form.

Level 3 — Google SSO: After Google OAuth callback, the backend generates Hydra tokens and then redirects to `/api/v1/auth/oauth/login`, which triggers the SSO Fast-Path for the full OIDC flow.

Q8.2: What is token introspection and why use it for SSO?

Answer: Token introspection (RFC 7662) is a server-to-server call to the OAuth2 provider (Hydra) asking: *"Is this token valid, and who does it belong to?"*

```
POST /admin/oauth2/introspect
token=<access_token>

Response: { "active": true, "sub": "kratos-uuid", "exp": 1720000000 }
```

Why not just decode the JWT locally?

- Access tokens may be opaque (not JWTs) depending on Hydra configuration.
- Introspection checks token **revocation** — a locally decoded JWT won't know if the token was revoked server-side (e.g., after logout).
- It provides the canonical source of truth for token validity.

---

9. Token Management & Lifecycle

Q9.1: What types of tokens does your system use?

Answer:

Token	Purpose	Storage	Lifetime	Format
Access Token	Authorizes API requests	HttpOnly cookie (path `/`)	Short-lived (configured in Hydra, typically 15-60 min)	Opaque or JWT
Refresh Token	Obtains new access tokens without re-login	HttpOnly cookie (path `/api/v1/auth`)	24 hours (cookie), Hydra manages server-side lifetime	Opaque
ID Token	Proves user identity (OIDC claim)	Never stored — validated once at callback and discarded	Single-use at callback	JWT (RS256-signed)

Q9.2: Why store tokens in HttpOnly cookies instead of localStorage or sessionStorage?

Answer:

Storage	XSS Vulnerable?	CSRF Vulnerable?
`localStorage`	✅ Yes — `document.cookie` equivalent via JS	❌ No
`sessionStorage`	✅ Yes	❌ No
HttpOnly Cookie	❌ No — JS cannot access	✅ Yes (mitigated by SameSite)

HttpOnly cookies are immune to XSS because JavaScript cannot read them. CSRF is mitigated by `SameSite=Lax` (browsers don't send cookies on cross-site POST requests) and the state parameter.

If a single XSS vulnerability exists, `localStorage`-based tokens can be exfiltrated to an attacker's server in milliseconds. With HttpOnly cookies, XSS can only make same-origin API calls (which SameSite + state help prevent).

Q9.3: Why scope `refresh_token` to `/api/v1/auth` but `access_token` to `/`?

Answer:

- **Access token** needs to be sent with every API request across the application, so it needs the broadest path (`/`).
- **Refresh token** is only needed for the token refresh endpoint (`/api/v1/auth/refresh`), so it's scoped to `/api/v1/auth`. This means:

- API requests to `/api/v1/users/profile` will include the `access_token` but NOT the `refresh_token`.
- If an attacker somehow intercepts requests to a non-auth endpoint, they only get the access token (short-lived), not the refresh token (long-lived).

---

10. Consent Flow & Hijack Prevention

Q10.1: What is a consent challenge and why do you auto-accept it?

Answer: In OAuth 2.0, after login, the user should be asked: *"Do you consent to this application accessing your data?"* This is the consent screen (like Google's "App X wants to access your email").

In our case, the auth service **is** the first-party application — the user is logging into SafeRide itself, not a third-party app. Asking them to consent to their own application is unnecessary UX friction. So we auto-accept:

```
redirectTo, err := h.orchestrator.AcceptConsentChallenge(
    ctx, challenge, consentRequest.Subject,
    []string{"openid", "offline_access"}, nil,
)
```

Q10.2: What is a "Consent Hijack" attack and how do you prevent it?

Answer: A **Consent Hijack** occurs when an attacker uses a valid `consent_challenge` from one user's session to gain tokens for another user.

Attack scenario:

1. Attacker starts OAuth flow → gets their own `login_challenge`.
2. Victim starts OAuth flow → gets their own `login_challenge`.
3. After victim completes login, Hydra issues a `consent_challenge`.
4. Attacker intercepts the `consent_challenge` and submits it from their own session.

Prevention:

```
// In Consent handler:
if session.LoginChallenge == "" || session.LoginChallenge ≠ consentRequest.LoginChallen
    return error("Consent hijack check failed: login challenge mismatch")
}
```

We verify that the `login_challenge` stored in the session (bound by `session_id` cookie) matches the `login_challenge` associated with the `consent_challenge` (returned by Hydra Admin API). The attacker's `session_id` would be bound to a different `login_challenge`, so the check fails.

11. CSRF, State & Nonce Protection

Q11.1: What is the `state` parameter and what attack does it prevent?

Answer: The `state` parameter is a **CSRF (Cross-Site Request Forgery) protection** for OAuth flows.

Attack (Login CSRF): An attacker initiates an OAuth flow with their own account and tricks the victim's browser into completing it. The victim unknowingly logs into the attacker's account, potentially entering sensitive data that the attacker can access.

Prevention: The backend generates a random `state` value (32 bytes of `crypto/rand`, hex-encoded) and stores it in the session:

```
state, err := generateRandomString(32)
```

At the callback, the `state` from the URL query is compared with the `state` in the session. The attacker cannot predict the victim's `state` value.

Q11.2: What is the `nonce` and how does it differ from `state`?

Answer:

Parameter	Protocol	Purpose	Verified Where
`state`	OAuth 2.0	CSRF protection for the redirect flow	At callback, against session DB
`nonce`	OIDC	**Replay protection** for the ID token	Inside the `id_token` JWT, at callback

The `nonce` is embedded in the `id_token` by Hydra. When we validate the `id_token`, we check:

```
h.orchestrator.ValidateIDToken(ctx, tokens.IDToken, session.Nonce, h.clientID)
```

If an attacker replays an old `id_token`, the nonce won't match the current session's nonce.

12. Session Fixation & Replay Attacks

Q12.1: What is a session fixation attack?

Answer: A session fixation attack occurs when an attacker **forces a known session ID onto a victim**. The attacker:

1. Creates a session (gets `session_id=abc123`).
2. Tricks the victim into using `session_id=abc123` (via URL manipulation, cookie tossing, etc.).
3. Victim authenticates using `session_id=abc123`.
4. Attacker uses `session_id=abc123` to access the now-authenticated session.

Mitigation: After successful OTP verification, we **regenerate the session ID**:

```
newSessionID := uuid.New()
// Create new session with same crypto material...
h.oauthSessionRepo.Create(ctx, newSession)     // create new
h.oauthSessionRepo.Delete(ctx, session.ID)      // delete old
h.setCookie(c, "session_id", newSessionID.String(), ...)
```

The old session ID (`abc123`) is deleted. The victim gets a new session ID. The attacker's known ID is invalid.

Q12.2: How do you prevent OAuth callback replay attacks?

Answer: Multiple layers:

1. **`used` flag:** `oauth_sessions.used` is set to `true` after the first callback. A replayed callback fails the `used == false` check.
2. **Session deletion:** After successful callback, the entire session is deleted from the DB.
3. **Nonce:** The `id_token`'s nonce is tied to a specific session. Replaying the same `id_token` with a different session's nonce fails.
4. **Authorization code single-use:** Hydra invalidates the authorization code after the first exchange. A replayed code returns an error.

```
// In Callback handler:
_ = h.oauthSessionRepo.MarkUsed(ctx, session.ID)
_ = h.oauthSessionRepo.Delete(ctx, session.ID)
h.deleteCookie(c, "session_id", "/api/v1/auth")
```

13. Open Redirect Prevention

Q13.1: What is an open redirect attack and how do you prevent it?

Answer: An **open redirect** attack tricks a user into clicking a legitimate-looking URL that redirects them to a malicious site. For example:

```
https://saferide.com/callback?redirect=https://evil.com/phish
```

Prevention: All redirect URLs are validated against a whitelist:

```
func (h *OAuthHandler) validateRedirectURL(redirectTo string) error {
    parsed, err := url.Parse(redirectTo)
    if err ≠ nil {
        return fmt.Errorf("invalid redirect URL")
    }
    // Allow relative paths (e.g., /api/v1/auth/oauth/login)
    if parsed.Host == "" {
        return nil
    }
    if !h.allowedHosts[parsed.Hostname()] {
        return fmt.Errorf("redirect to disallowed host: %s", parsed.Hostname())
    }
    return nil
}
```

The whitelist is built at startup from:

- Configured allowed redirect domains
- Frontend URL's host (automatically added)
- Hydra browser URL's host (automatically added)

Every redirect from Hydra Admin API responses is validated before the browser is redirected:

```
if valErr := h.validateRedirectURL(redirectTo); valErr ≠ nil {
    c.String(http.StatusBadRequest, "Redirect URL validation failed")
    return
}
```

14. Distributed System Compensation (Saga)

Q14.1: Why do you need a compensation/saga pattern in registration?

Answer: Registration involves **three distributed systems** that don't share a single transaction:

1. **Ory Kratos** — Create identity (HTTP API call)
2. **Ory Keto** — Create relationship tuple (HTTP API call)
3. **PostgreSQL** — Create user record + delete pending auth + publish event (DB transaction)

If the Postgres transaction fails **after** the Kratos identity was created, we'd have an orphaned identity in Kratos that nobody references. This violates data consistency.

Solution — Compensating Transaction:

```
compensate := func() {
    if isNewIdentity {
        _ = s.orchestrator.DeleteIdentity(ctx, kratosID)
    }
}

// ... if any DB operation fails ...
tx.Rollback()
compensate()  // Clean up Kratos
```

Note: Compensation only runs for **newly created** identities. If we reused an existing identity (same email already in Kratos), we don't delete it — other users/services may depend on it.

Q14.2: What if the compensation itself fails (Kratos DELETE fails)?

Answer: This is the classic **dual-write problem**. In our implementation, we log the error but don't retry. In production, you could:

1. **Store compensation in a "dead letter" table** — a background worker retries periodically.
2. **Use the Outbox pattern** — publish a `CompensationRequired` event to Kafka, consumed by a reliable worker.
3. **Reconciliation job** — periodically compare Kratos identities with local DB users and remove orphans.

For SafeRide, the risk is low because orphaned Kratos identities have no access (no user record = no device ownership = no API access).

15. Rate Limiting & Brute Force Protection

Q15.1: How is rate limiting implemented?

Answer: IP-based rate limiting using Redis:

```
func RateLimiter(cacheProvider cache.CacheProvider, limit int, window time.Duration, key
    return func(c *gin.Context) {
        ip := c.ClientIP()
        key := fmt.Sprintf("auth:rate:%s:%s", keyPrefix, ip)
        count, err := cacheProvider.Increment(ctx, key)

        if count == 1 {
            _ = cacheProvider.Set(ctx, key, count, window)  // Set TTL on first request
        }

        if count > int64(limit) {
            c.Header("Retry-After", fmt.Sprintf("%d", int(window.Seconds())))
            c.Abort()  // Block request
            return
        }
        c.Next()
    }
}
```

Design decisions:

- **Redis-based** — works across multiple backend instances (horizontal scaling).
- **IP-based key** — keyed by `auth:rate:<prefix>:<clientIP>`.
- **Sliding window via TTL** — the first request in a window sets the TTL. Subsequent requests within that window increment the counter.
- **Graceful degradation** — if Redis is down (`err ≠ nil`), the request is allowed through (`c.Next()`). This prevents Redis failures from blocking all authentication.
- **Retry-After header** — RFC-compliant response telling clients how long to wait.

Q15.2: What are the different levels of brute-force protection?

Answer:

Layer	Scope	Mechanism
Rate Limiter	Per IP, per endpoint	Redis counter with TTL window
OTP Attempts	Per session	`pending_auth.attempt_count` with max 5
OTP Expiry	Per session	5-minute TTL; expired OTPs rejected
Session Deletion	Per session	On max attempts exceeded, session is deleted
Constant-time Compare	Per OTP check	Prevents timing side-channel attacks
CSPRNG OTP	Per OTP	`crypto/rand` prevents OTP prediction

---

16. Security Headers

Q16.1: What security headers does the service set and why?

Answer:

```
c.Header("Strict-Transport-Security", "max-age=31536000; includeSubDomains; preload")
c.Header("X-Content-Type-Options", "nosniff")
c.Header("X-Frame-Options", "DENY")
c.Header("X-XSS-Protection", "1; mode=block")
c.Header("Referrer-Policy", "strict-origin-when-cross-origin")
c.Header("Content-Security-Policy", "default-src 'self'; ...")
```

Header	Purpose
HSTS	Forces HTTPS for 1 year. `includeSubDomains` covers all subdomains. `preload` allows inclusion in browser HSTS preload lists — even the first visit is HTTPS.
X-Content-Type-Options: nosniff	Prevents MIME type sniffing. Stops browsers from interpreting a JSON response as HTML (XSS vector).
X-Frame-Options: DENY	Prevents clickjacking by blocking the page from being embedded in `<iframe>`.
X-XSS-Protection	Enables browser's built-in XSS filter (legacy, but defense-in-depth).
Referrer-Policy	Prevents leaking full URLs (including tokens/state in query strings) to external sites. `strict-origin-when-cross-origin` sends only the origin for cross-origin requests.
CSP	Whitelist-based control over which scripts, styles, fonts, and connections the browser can load. `frame-ancestors 'none'` is the modern equivalent of X-Frame-Options.

---

17. Google OAuth / Social Login

Q17.1: How does Google OAuth integration work?

Answer:

1. **Login Initiation:** Backend generates a cryptographic `state` token, stores it in a short-lived HttpOnly cookie (`oauthstate`, 10 min), and redirects to Google's authorization endpoint.
2. **Callback:**
   - **CSRF Validation:** Read `oauthstate` cookie, immediately delete it (one-time use), compare with `state` query parameter using constant-time comparison.
   - **Code Exchange:** Exchange authorization code for Google tokens with a 10-second timeout.
   - **User Info Fetch:** Call Google's userinfo API using the OAuth2 HTTP client. Response body limited to 1MB (`io.LimitReader`).
   - **Email Validation:** Ensure email is present and verified by Google.
   - **Auto-Registration:** If user doesn't exist, auto-create in Kratos + local DB within a transaction with compensation.
   - **Token Generation:** Generate Hydra tokens using the user's `kratosID`.

- **SSO Integration:** Set HttpOnly token cookies and redirect to `/api/v1/auth/oauth/login` for seamless SSO integration.

Q17.2: Why do you use a 10-second timeout for Google code exchange?

Answer:

```
ctx, cancel := context.WithTimeout(c.Request.Context(), 10*time.Second)
defer cancel()
token, err := h.oauthConfig.Exchange(ctx, code)
```

This prevents **Slowloris-style resource exhaustion**. If Google's servers are slow or unresponsive, without a timeout, the goroutine would hang indefinitely, eventually exhausting server resources under load. The 10-second timeout ensures requests fail fast and free up resources.

Q17.3: Why do you limit the Google userinfo response to 1MB?

Answer:

```
contents, err := io.ReadAll(io.LimitReader(resp.Body, 1<<20))
```

This prevents **memory exhaustion attacks**. If Google's API was compromised (or a MITM attacker replaced the response), an unbounded `io.ReadAll` could attempt to allocate gigabytes of memory, crashing the service. The 1MB limit is far more than Google userinfo needs (~500 bytes) but protects against adversarial responses.

---

18. Refresh Token Rotation & Grace Period

Q18.1: What is refresh token rotation and why is it important?

Answer: Refresh token rotation means that every time a refresh token is used, it's **invalidated** and a **new refresh token** is issued alongside the new access token. This limits the damage of a stolen refresh token — the attacker's copy becomes invalid after the legitimate user refreshes.

Q18.2: What is the "grace period" and what problem does it solve?

Answer: The **grace period** solves the **concurrent refresh race condition**.

Problem: A single-page app might make multiple API calls simultaneously. If the access token expires, multiple requests trigger concurrent refresh calls. With rotation, only the first refresh succeeds — the second uses the old (now-invalidated) refresh token and fails, logging the user out.

Solution:

```
// Check Redis cache for grace period result
cacheKey := "grace_period:refresh:" + refreshToken
cachedVal, err := h.cacheProvider.Get(ctx, cacheKey)
if err == nil && cachedVal != "" {
    // Return the already-rotated tokens from cache
    return cached tokens
}

// Perform actual rotation with Hydra
res, err := h.refreshService.Refresh(ctx, refreshToken)

// Cache result for 10 seconds under the OLD token key
_ = h.cacheProvider.Set(ctx, cacheKey, serialized, 10*time.Second)
```

The first request performs the actual rotation and caches the result for 10 seconds. Concurrent requests using the same old refresh token hit the cache and receive the same new tokens — no double-rotation, no logout.

---

19. Ory Stack (Kratos, Hydra, Keto)

Q19.1: Why use the Ory stack instead of building auth from scratch or using Auth0/Firebase?

Answer:

Factor	Ory (Self-Hosted)	Auth0/Firebase	Custom Built
Data Sovereignty	✅ Full control, self-hosted	❌ Data stored in vendor's cloud	✅ Full control
Vendor Lock-in	✅ Open source, no lock-in	❌ Proprietary APIs	✅ No lock-in
Cost at Scale	✅ Fixed infrastructure cost	❌ Per-MAU pricing escalates rapidly	✅ Fixed cost
Compliance	✅ Deploy in any region	⚠️ Limited region control	✅ Full control
Security Audit	✅ Battle-tested, open source	⚠️ Black box	❌ High risk of vulnerabilities
Development Time	✅ Pre-built, standards-compliant	✅ Fastest	❌ Months of development
Flexibility	✅ Full customization	⚠️ Limited by vendor features	✅ Full customization

---

Q19.2: What is the role of each Ory component?

Answer:

Component	Role	API Examples
Ory Kratos	**Identity Management** — stores user identities (email, schemas, traits)	`POST /admin/identities`, `GET /admin/identities?credentials_identifier=email`
Ory Hydra	**OAuth2/OIDC Provider** — issues tokens, handles authorization flows, manages consent	`PUT /admin/oauth2/auth/requests/login/accept`, `POST /oauth2/token`, `POST /admin/oauth2/introspect`
Ory Keto	**Authorization (ReBAC)** — relationship-based access control (Google Zanzibar-style)	Create relation tuples like `devices/SR-2024-00142/owner = kratos-uuid`

The separation means:

- Kratos knows **who** the user is.
- Hydra knows **how** to issue tokens (OAuth2 standard).
- Keto knows **what** the user can do (permissions).

Q19.3: What is Ory Keto's relationship-based access control?

Answer: Keto implements **Google Zanzibar**-style authorization using **relation tuples**:

```
namespace:object#relation@subject
```

In our system:

```
devices:SR-2024-00142#owner@kratos-uuid     ← User owns this device
platform:platform#admin@kratos-uuid          ← User is platform admin
```

Why ReBAC over RBAC?

- **RBAC (Role-Based):** "User X has role ADMIN" → flat, hard to model fine-grained permissions.
- **ReBAC (Relationship-Based):** "User X is the owner of Device Y" → naturally models ownership, delegation, hierarchical permissions.

For SafeRide, device ownership is central — a user should only control their own devices. ReBAC directly models this relationship.

---

20. Outbox Pattern & Event-Driven Design

Q20.1: What is the Outbox Pattern and why do you use it?

Answer: The Outbox Pattern solves the **dual-write problem** — when you need to both update a database and publish a message to a broker (Kafka), and you need **exactly-once semantics**.

Problem without Outbox:

```
1. INSERT INTO users ( ... ) → ✅ Success
2. kafka.Publish("UserRegistered") → ❌ Fails (network error)
→ Database has user, but no event was published. Downstream services are inconsistent.
```

Solution with Outbox:

```
1. BEGIN TRANSACTION
2.   INSERT INTO users ( ... )
3.   INSERT INTO outbox_events (event_type, payload, ... )
4. COMMIT
→ Both writes succeed or both fail (ACID guarantee).
5. Background worker polls outbox_events → publishes to Kafka → marks as processed.
```

In our code:

```
ctxTx := eventbus.WithTx(ctx, tx)

s.userRepo.Create(ctxTx, u)         // Same transaction
s.eventBus.Publish(ctxTx, evt)      // Same transaction (writes to outbox table)

tx.Commit()                          // Atomic commit
```

Q20.2: What events does the auth service publish?

Answer:

Event	When	Payload
`OTPRequested`	Registration/login initiated	`{type, email, otp}` — triggers email sending
`UserRegistered`	Successful registration	`{kratos_id, email, display_name, device_id}` — triggers downstream setup
`UserLoggedIn`	Successful login	`{kratos_id, email}` — for audit/analytics

All events include `correlation_id` for distributed tracing.

---

21. Database & Migration Design

Q21.1: What tables does the auth service use?

Answer:

Table	Purpose	Key Columns
`users`	Local user records	`id (UUID)`, `kratos_id`, `email`, `display_name`, `phone_number`, `is_active`
`pending_auth`	Temporary OTP sessions	`id (UUID)`, `email`, `otp_hash`, `purpose`, `expires_at`, `attempt_count`, `max_attempts`, `metadata`
`outbox_events`	Outbox pattern for reliable event publishing	`id`, `event_type`, `payload`, `aggregate_id`, `aggregate_type`, `correlation_id`
`audit_logs`	Security event logging	`id`, `action`, `details`
`idempotency_keys`	Prevent duplicate processing	`key`, `response`, `expires_at`
`oauth_sessions`	OIDC flow state (state, nonce, PKCE, login_challenge)	`id (UUID)`, `state`, `nonce`, `code_verifier`, `login_challenge`, `used`, `expires_at`

Q21.2: Why store OAuth sessions in PostgreSQL instead of Redis?

Answer: OAuth sessions contain **critical security material** (PKCE code_verifier, state, nonce). PostgreSQL provides:

- **Durability** — Redis is typically configured without persistence or with periodic snapshots. A Redis restart loses sessions. PostgreSQL guarantees ACID durability.
- **Queryability** — We can query by state, login_challenge, or session ID with indexed lookups.
- **Consistency** — The `used` flag and `login_challenge` binding are protected by DB transactions.

Registration sessions, on the other hand, use Redis because they're short-lived (5 min), don't contain cryptographic material, and benefit from Redis's sub-millisecond reads.

---

22. Logout & Token Revocation

Q22.1: How does logout work?

Answer:

```
func (h *LogoutHandler) Logout(c *gin.Context) {
    refreshToken, _ := c.Cookie("refresh_token")
    kratosID := c.GetHeader("X-User-Id")
    corrID := c.GetString("correlation_id")

    err := h.logoutService.Logout(ctx, refreshToken, kratosID, corrID)

    c.SetCookie("refresh_token", "", -1, "/api/v1/auth", ...)  // Delete
    c.SetCookie("access_token", "", -1, "/", ...)               // Delete
    c.SetCookie("login_challenge", "", -1, "/", ...)            // Delete
    c.Status(http.StatusNoContent)
}
```

Logout performs:

1. **Server-side revocation** — Calls Hydra to revoke the refresh token (and by extension, all access tokens issued from it).
2. **Cookie deletion** — Clears all auth cookies by setting `maxAge=-1`.
3. **Event publishing** — Publishes logout event for audit logging.

Q22.2: Why is simply deleting cookies not sufficient for logout?

Answer: Deleting cookies only prevents the **browser** from sending tokens. If an attacker has already exfiltrated the tokens (e.g., via a network MITM or log exposure), they can still use them. **Server-side revocation** tells Hydra to reject the token even if someone presents it directly. Introspection will return `active: false` for revoked tokens.

23. Attack Scenarios & Mitigations

Q23.1: Summary of all attack vectors and their mitigations

#	Attack	Mitigation	Code Reference
1	**ID Token Forgery**	RS256 signature verification via JWKS	`ValidateIDToken()` in Callback
2	**Fake Login Challenge**	Verify challenge via Hydra Admin API before use	`GetLoginRequest()` in LoginChallenge
3	**Session Fixation**	Regenerate session_id after successful OTP	`uuid.New()` in VerifyOTP
4	**Authorization Code Interception**	PKCE S256 — verifier never leaves server	`ExchangeCodeForTokens(code, verifier)`
5	**CSRF / Login CSRF**	`state` parameter + `SameSite=Lax` cookies	State validation in Callback
6	**ID Token Replay**	Nonce verification inside id_token	`ValidateIDToken(nonce)`
7	**Open Redirect**	URL whitelist validation	`validateRedirectURL()`
8	**OTP Brute Force**	Attempt limiting + rate limiting + expiry	`HasExceededAttempts()` + RateLimiter
9	**Timing Attack on OTP**	`crypto/subtle.ConstantTimeCompare`	`VerifyOTP()`
10	**Consent Hijack**	Session-bound login_challenge check	`session.LoginChallenge == consent.LoginChallenge`
11	**Token Theft via XSS**	HttpOnly cookies	`setCookie(httpOnly=true)`

#	Attack	Mitigation	Code Reference
12	**Registration Session Hijack**	Double-binding check (cookie + metadata)	`meta.RegistrationSessionID ≠ req.RegistrationSessionID`
13	**Callback Replay**	`used` flag + session deletion + code single-use	`MarkUsed()` + `Delete()`
14	**Clickjacking**	`X-Frame-Options: DENY` + CSP `frame-ancestors 'none'`	SecurityHeaders middleware
15	**MIME Sniffing**	`X-Content-Type-Options: nosniff`	SecurityHeaders middleware
16	**Downgrade to HTTP**	HSTS with preload	SecurityHeaders middleware
17	**Orphaned Kratos Identity**	Saga compensation pattern	`compensate()` on tx failure
18	**Refresh Token Race Condition**	10-second Redis grace period cache	RefreshHandler cache
19	**Memory Exhaustion (Google SSO)**	`io.LimitReader(1MB)`	GoogleCallback
20	**Google State CSRF**	Cookie-stored state + constant-time compare	`secureCompare()`

Q23.2: What would happen if someone stole both the authorization code AND the state parameter?

Answer: They **still cannot** complete the flow because:

1. **PKCE** — The `code_verifier` is server-side. Without it, Hydra rejects the token exchange.
2. **session_id cookie** — The callback handler reads the `session_id` cookie and finds the session in the DB. The attacker's browser doesn't have this HttpOnly cookie.
3. **Even if** they somehow craft the cookie, the session is bound to the DB record containing the `code_verifier`, which they don't know.

The attacker would need: authorization code + state + session_id cookie + code_verifier — and the latter two never leave the server/cookie.

Q23.3: What if an attacker compromises your Redis instance?

Answer: Redis stores:

- Registration session state (`{kratos_id, verified, authenticated}`)
- Rate limiting counters
- Refresh token grace period cache

Impact:

- **Registration sessions** — Attacker could create a fake verified session, but they still need the `registration_session_id` HttpOnly cookie to use it. They'd also need to bypass the OIDC flow (Hydra challenge, PKCE, etc.).
- **Rate limits** — Attacker could reset rate limits. Mitigated by OTP attempt counting (in PostgreSQL).
- **Grace period cache** — Attacker could see tokens temporarily cached. Mitigated by the 10-second TTL and the fact that tokens are also stored in HttpOnly cookies.

Recommendation: Redis should be deployed with:

- TLS-encrypted connections
- AUTH password
- Network isolation (private VPC, no public access)
- No `KEYS` / `FLUSHALL` commands in production

---

24. Production Readiness

Q24.1: What security measures differentiate production from development?

Answer:

```
isProduction bool

// In setCookie:
c.SetCookie(name, value, maxAge, path, "", h.isProduction, true)
//                                         ^^^^^^^^^^^^^^^^
//                                         Secure=true only in prod
```

Feature	Development	Production
Cookie `Secure` flag	`false` (allow HTTP for localhost)	`true` (HTTPS only)
HSTS header	Set but ineffective on HTTP	Enforces HTTPS for 1 year
CSP header	Same strict policy	Same strict policy
Rate limiting	Active	Active (same config)

Q24.2: What would you add for a truly production-hardened system?

Answer:

1. **JWT-based access tokens** with local signature verification (reduces Hydra introspection load).
2. **Token binding (DPoP)** — bind tokens to a client's cryptographic key, so stolen tokens can't be used from other devices.
3. **Audit logging to Kafka** — every auth event (login, logout, OTP failure) published for SIEM analysis.
4. **Anomaly detection** — flag impossible travel, simultaneous sessions from different geolocations.
5. **Hardware security module (HSM)** — for signing key storage instead of software-based JWKS.
6. **Mutual TLS (mTLS)** — between backend services and Ory components.
7. **WAF (Web Application Firewall)** — block common attack patterns at the edge.
8. **OCSP stapling** — for TLS certificate revocation checking.
9. **Secret rotation** — automated rotation of Hydra client secrets, Redis passwords, DB credentials.
10. **Penetration testing** — regular third-party security audits.

Q24.3: How do you handle horizontal scaling of the auth service?

Answer: The auth service is **stateless** by design:

- **Sessions** are in PostgreSQL (shared across instances).
- **Registration/login state** is in Redis (shared across instances).
- **No in-memory state** — each request is self-contained.
- **Rate limiting** uses Redis (shared counters work across instances).
- **Sticky sessions** are NOT required — any instance can handle any request because all state is externalized.

You can scale horizontally by adding more auth service pods behind a load balancer. The only bottleneck is the database, which can be addressed with read replicas and connection pooling (PgBouncer).

> **Last Updated:** July 2026 | **Based on:** SafeRide Auth Service codebase analysis